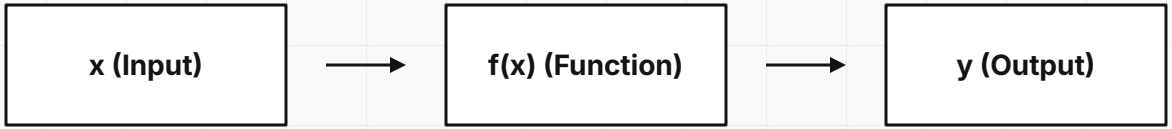
Indrajeet Haldar

Architect / Technologist

Senior Machine Learning Engineer @ Autodesk
(tldr: lead some of the post training work for autodesk's foundational models)

AI in Architecture

AI in Architecture



Presentation Outline

Overview:

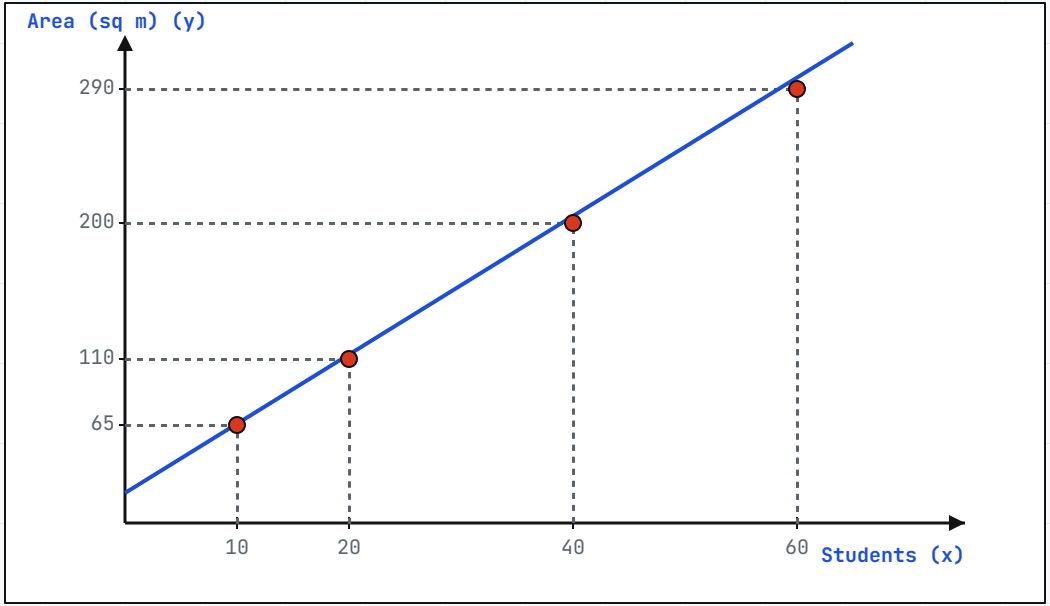
1. How LLM's work: Polynomial Functions to Transformers
2. AI Integration in CAD and context engineering
3. Demos: Rhino MCP, Grasshopper, and Autodesk
4. Broader trends
5. Questions and Answers

1. How LLMs work

A Basic Polynomial Function: y = mx + c

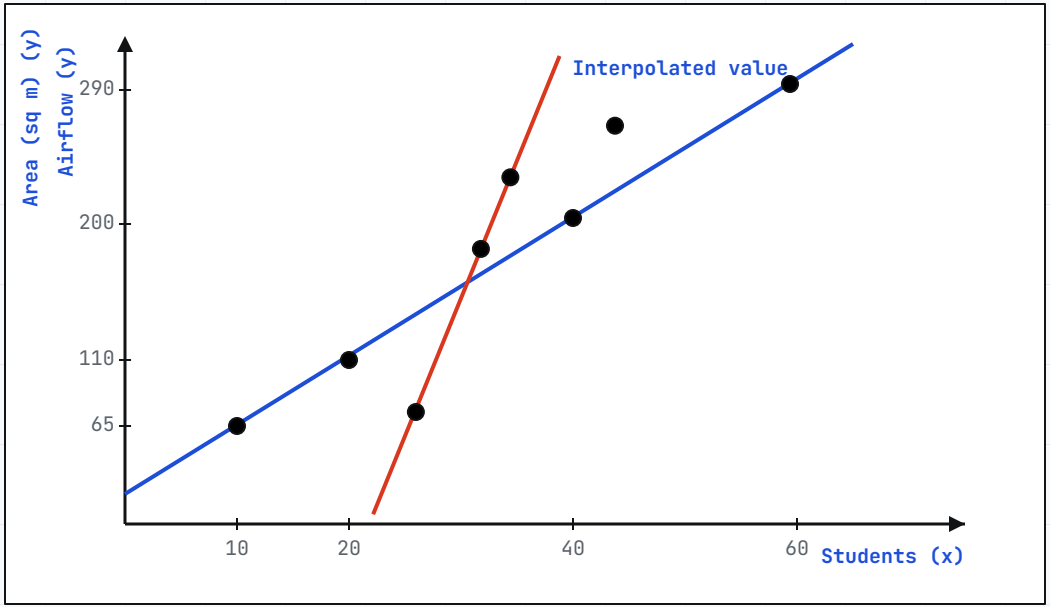
`y = 4.5x + 20` | x = student count, m = 4.5 sq m/student, c = 20 sq m buffer, y = studio area

Students (x)	Area (sq m) (y)
10	65
20	110
40	200
60	290



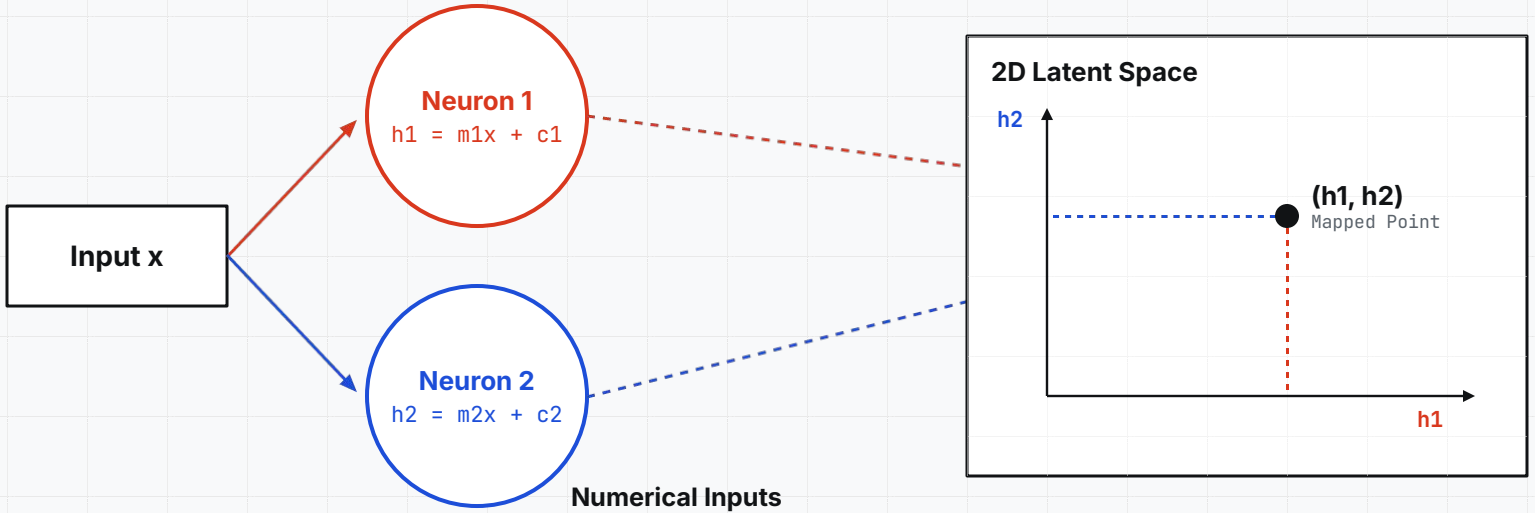
Ref: Legendre (1805); synthetic illustrative dataset.

Two Functions



Note: Simplified mathematical representation shown.

Two Dense Neurons



Numerical Inputs

Instead of just singular numbers - you can plug in a stack of numbers (a vector) to get a vector out as well

Ref: Rosenblatt (1958); Rumelhart et al. (1986).

Word Vectorization

Tokenization divides text into words. Vectorization assigns numerical coordinates to words.

STEP 01

Raw Text

"Studio Room"

Raw Input String

Text Input

Natural language consists of character sequences. Numerical models process coordinate arrays rather than text strings.

Appendix: Mathematical formulation of vectorization.

Word Vectorization

Tokenization divides text into words. Vectorization assigns numerical coordinates to words.

STEP 01

Raw Text

"Studio Room"

Raw Input String

tokenize

STEP 02 : SEGMENT

Tokenization

Token 1: "Studio"

Token 2: "Room"

Text Input

Natural language consists of character sequences. Numerical models process coordinate arrays rather than text strings.

Word Tokens

Text is divided into discrete word units. Each word corresponds to an index entry.

Appendix: Mathematical formulation of vectorization.

Word Vectorization

Tokenization divides text into words. Vectorization assigns numerical coordinates to words.

STEP 01

Raw Text

"Studio Room"

Raw Input String

tokenize

STEP 02 : SEGMENT

Tokenization

Token 1: "Studio"

Token 2: "Room"

vectorize

STEP 03 : ENCODE

Vectorization

"Studio" → [0.42, 0.85]

"Room" → [0.78, 0.31]

Text Input

Natural language consists of character sequences. Numerical models process coordinate arrays rather than text strings.

Word Tokens

Text is divided into discrete word units. Each word corresponds to an index entry.

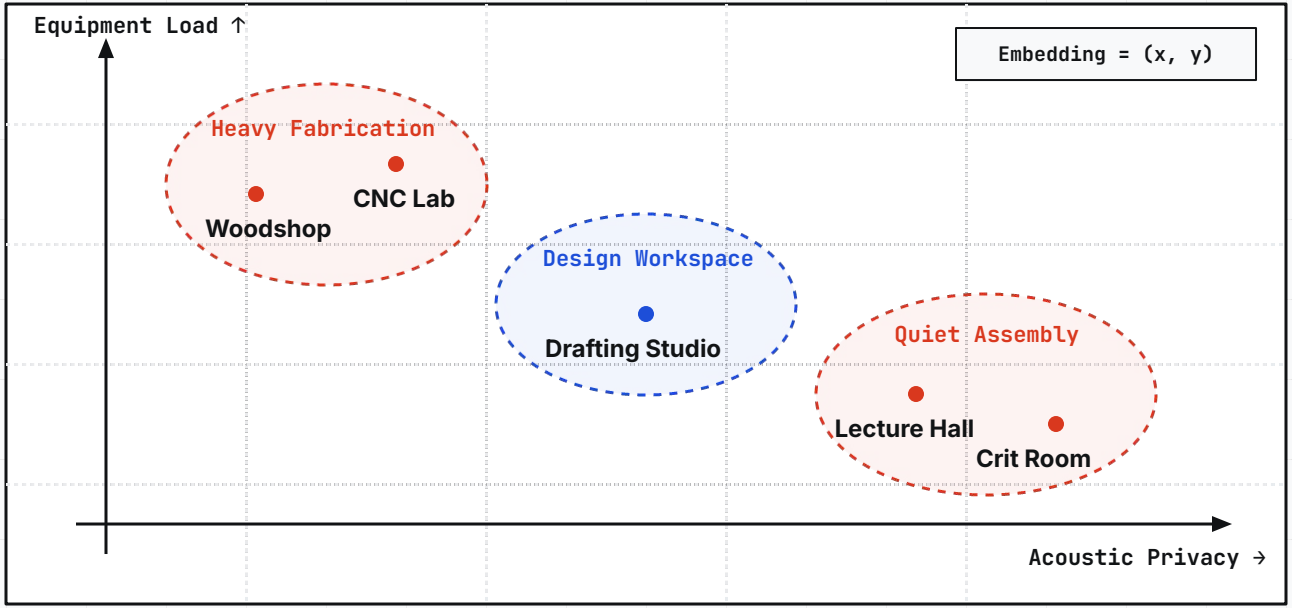
Coordinate Vectors

Each word token corresponds to coordinates in a continuous vector space. Distance indicates statistical correlation.

Appendix: Mathematical formulation of vectorization.

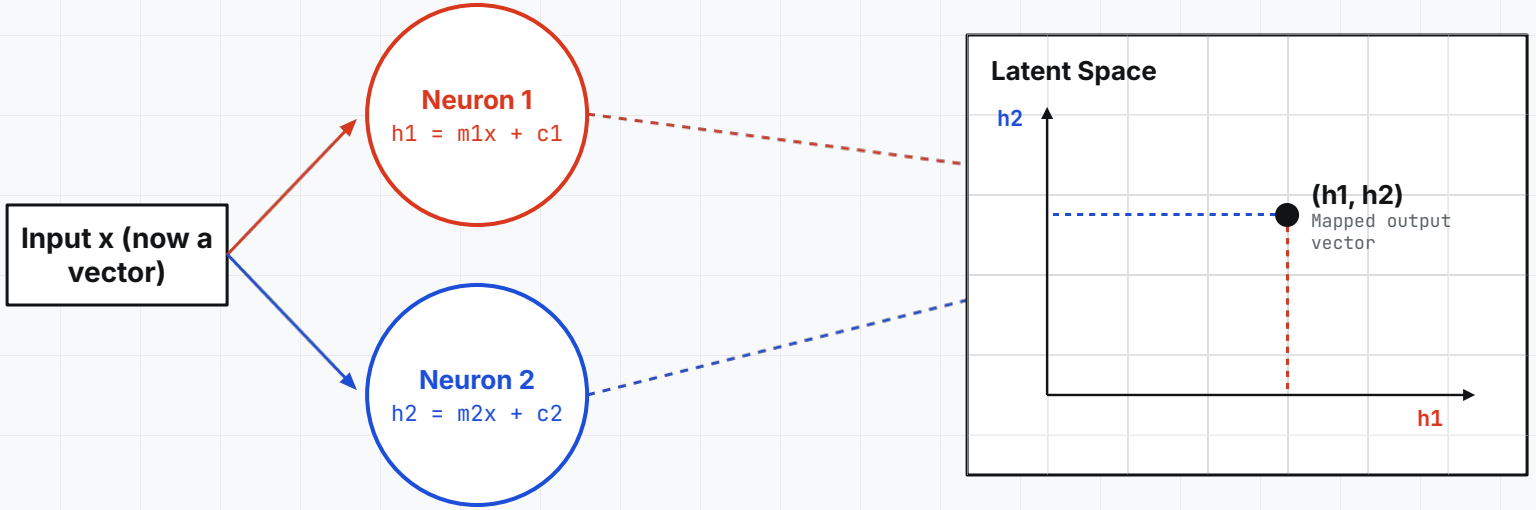
Vector Embeddings and Latent Space

Words and spatial room labels map to coordinate values. Proximity corresponds to mathematical correlation.



Ref: Mikolov et al. (2013), Word2Vec.

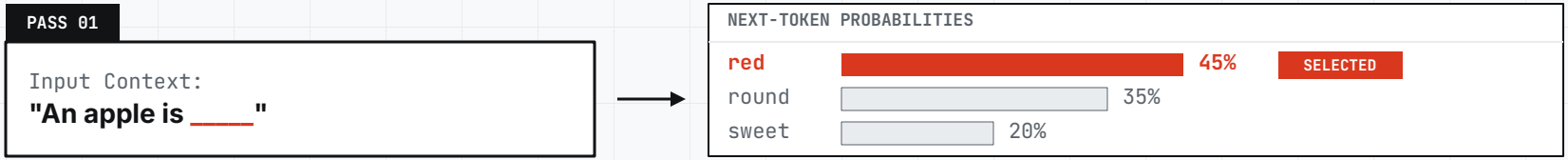
Next-Token Prediction Using Vectors



Ref: Rosenblatt (1958); Rumelhart et al. (1986).

Sequential Next-Token Prediction (Autoregression and Recurrent Models)

Sequential models store preceding token states in a fixed memory vector.



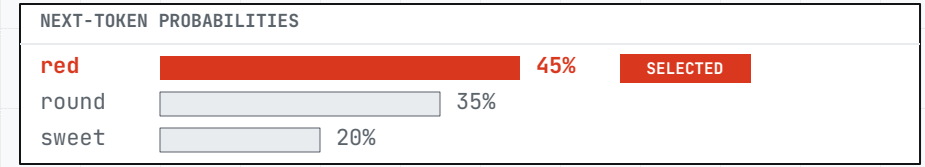
Ref: Hochreiter & Schmidhuber (1997).

Sequential Next-Token Prediction (Autoregression and Recurrent Models)

Sequential models store preceding token states in a fixed memory vector.

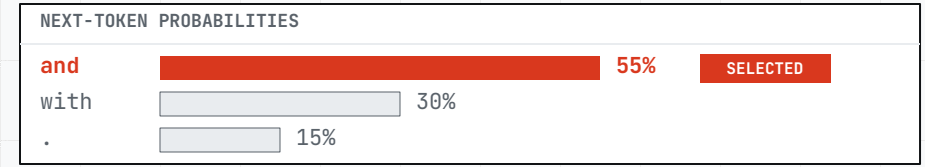
PASS 01

Input Context:
"An apple is _____"



PASS 02

Input Context (fed 'red' back):
"An apple is red _____"



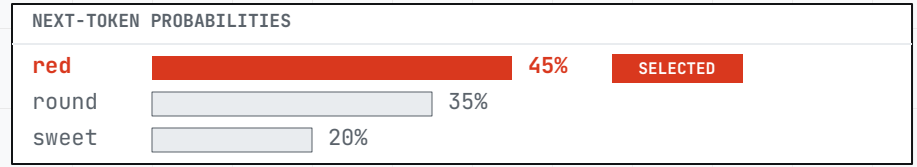
Ref: Hochreiter & Schmidhuber (1997).

Sequential Next-Token Prediction (Autoregression and Recurrent Models)

Sequential models store preceding token states in a fixed memory vector.

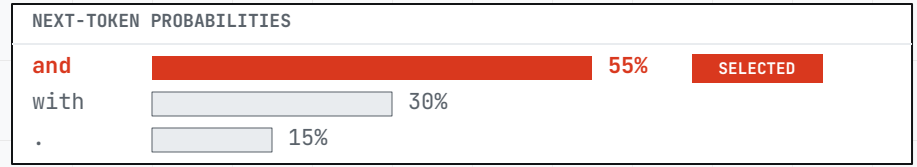
PASS 01

Input Context:
"An apple is _____"



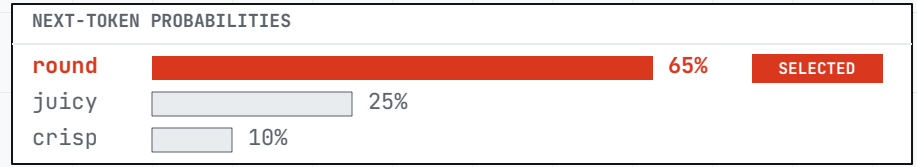
PASS 02

Input Context (fed 'red' back):
"An apple is red _____"



PASS 03

Input Context (fed 'and' back):
"An apple is red and _____"

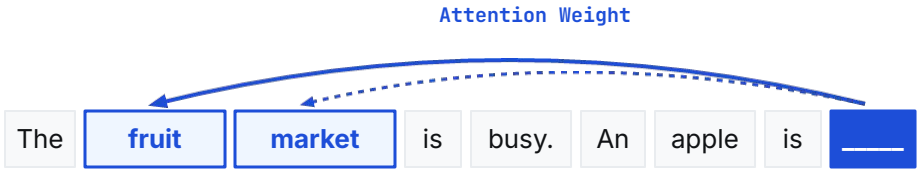


AUTOREGRESSIVE LOOP: Output tokens are iteratively appended to input sequences to generate subsequent tokens sequentially.

Ref: Hochreiter & Schmidhuber (1997).

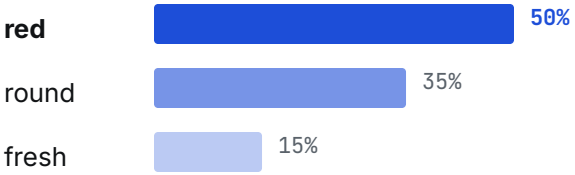
Transformers: Conditional Probabilities Based on Context

CONTEXT A: FRUIT MARKET

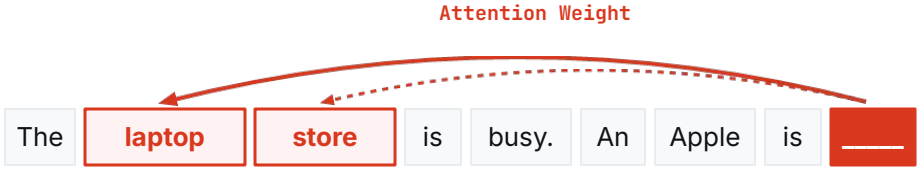


Prompt: "The fruit market is busy. An apple is _____"

NEXT-WORD PROBABILITIES P(w | Context)

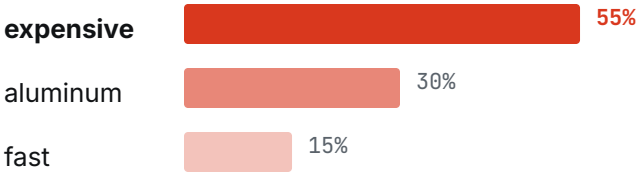


CONTEXT B: LAPTOP STORE



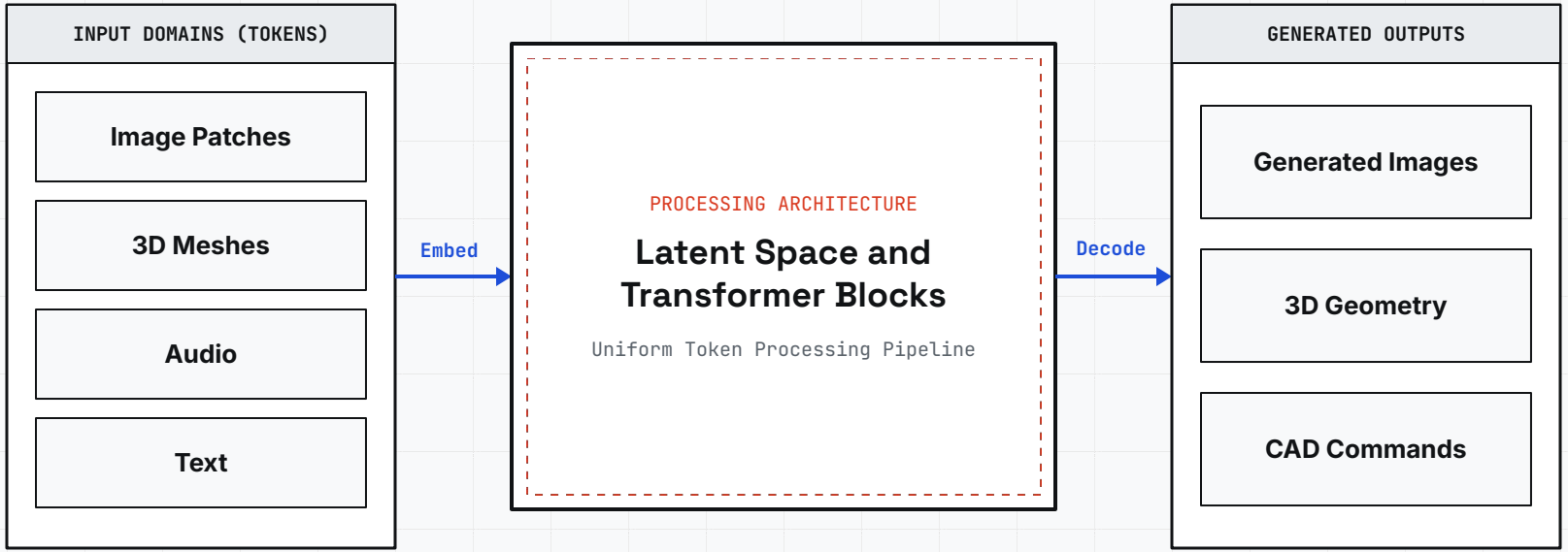
Prompt: "The laptop store is busy. An Apple is _____"

NEXT-WORD PROBABILITIES P(w | Context)



Ref: Vaswani et al. (2017), Attention Is All You Need.

Images, 3D Models, and Other Media



Ref: Dosovitskiy et al. (2020), Vision Transformer.

System Prompts and Model Interaction Structure

USER INTERFACE VIEW

Interactive chat interface.



Text Interface Display

INPUT DATA TO MODEL

[Hidden System Prompt]

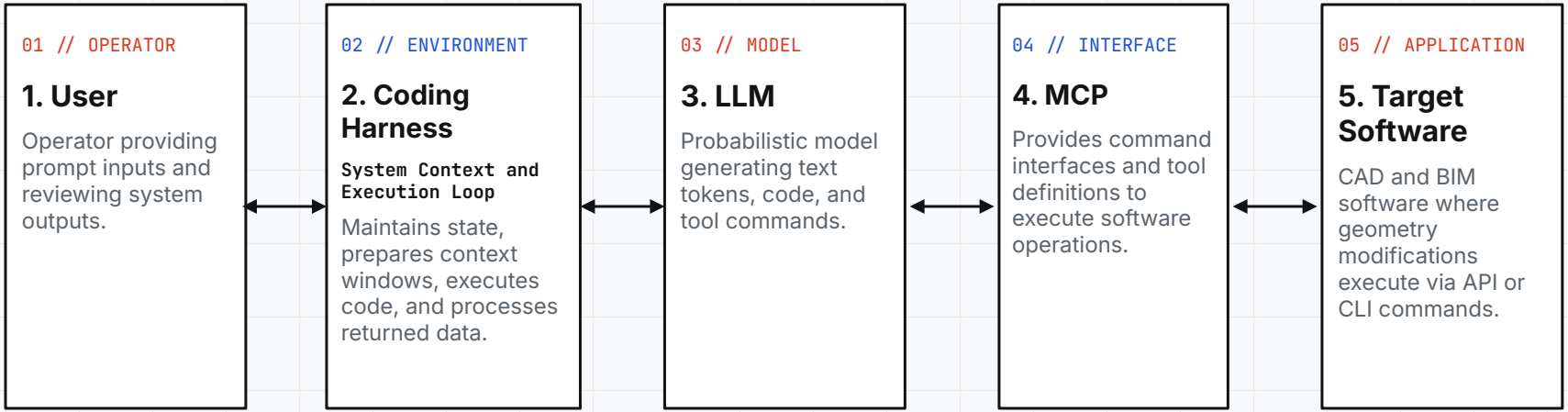
+ [Relevant Files & CAD State Selected by Harness]

+ [User Prompt]

→ [Token Probability Calculation]

2. AI Integration in CAD and context engineering

General Software Architecture: LLMs, Coding Harnesses, and MCPs



Ref: Anthropic (2024), Model Context Protocol.

Live Demonstrations: CAD Automation via MCP

SECTION 03 // INTERACTIVE EXECUTION WORKFLOWS

DEMO 01 // GEOMETRY

Rhino 8 Model Context

Direct Viewport & Brep Control

- **Native Viewport API:** Inspect active layers, materials, and selected geometry.
- **Command Execution:** Automated NURBS curves, lofting, sweeps, and booleans.
- **Bi-Directional State:** Streams model bounding boxes and object GUIDs to LLM.

PROTOCOL: JSON-RPC (STDIO)

Ref: Anthropic Model Context Protocol (MCP); McNeel RhinoAI Integration.

Live Demonstrations: CAD Automation via MCP

SECTION 03 // INTERACTIVE EXECUTION WORKFLOWS

DEMO 01 // GEOMETRY

Rhino 8 Model Context

Direct Viewport & Brep Control

- **Native Viewport API:** Inspect active layers, materials, and selected geometry.
- **Command Execution:** Automated NURBS curves, lofting, sweeps, and booleans.
- **Bi-Directional State:** Streams model bounding boxes and object GUIDs to LLM.

PROTOCOL: JSON-RPC (STDIO)

DEMO 02 // PARAMETRIC

Grasshopper Nodes

Algorithmic Graph Modification

- **Graph Inspection:** Reads active sliders, panel parameters, and cluster data.

ENGINE: RHINOCODE PYTHON 3

Ref: Anthropic Model Context Protocol (MCP); McNeel RhinoAI Integration.

Live Demonstrations: CAD Automation via MCP

SECTION 03 // INTERACTIVE EXECUTION WORKFLOWS

DEMO 01 // GEOMETRY

Rhino 8 Model Context

Direct Viewport & Brep Control

- **Native Viewport API:** Inspect active layers, materials, and selected geometry.
- **Command Execution:** Automated NURBS curves, lofting, sweeps, and booleans.
- **Bi-Directional State:** Streams model bounding boxes and object GUIDs to LLM.

PROTOCOL: JSON-RPC (STDIO)

DEMO 02 // PARAMETRIC

Grasshopper Nodes

Algorithmic Graph Modification

- **Graph Inspection:** Reads active sliders, panel parameters, and cluster data.

ENGINE: RHINOCODE PYTHON 3

DEMO 03 // ECOSYSTEM

Autodesk & Interop

Cross-Platform Pipeline

- **Forma**

PIPELINE: HYBRID CAD / BIM

Ref: Anthropic Model Context Protocol (MCP); McNeel RhinoAI Integration.

Demo 1: Rhino MCP

Installing the Rhino MCP (mcneel.github.io/RhinoAI)

SETUP GUIDE // RHINO 8 AND CODEX

STEP 01 // ENVIRONMENT

Rhino 8

Use current Rhino 8 on an Apple Silicon Mac.

CLI / COMMAND

```
# In Rhino 8 command line:
SystemInfo
# Check the Rhino version
```

Tested here: Rhino 8.36 on Apple Silicon.

Ref: McNeel RhinoAI (https://mcneel.github.io/RhinoAI/); Anthropic Claude Desktop MCP Documentation.

Installing the Rhino MCP (mcneel.github.io/RhinoAI)

SETUP GUIDE // RHINO 8 AND CODEX

STEP 01 // ENVIRONMENT

Rhino 8

Use current Rhino 8 on an Apple Silicon Mac.

CLI / COMMAND

```
# In Rhino 8 command line:
SystemInfo
# Check the Rhino version
```

Tested here: Rhino 8.36 on Apple Silicon.

STEP 02 // PACKAGE

Install MCP Server

Install Rhino-MCP-Platform from Rhino Package Manager.

RHINO COMMAND

```
PackageManager
# Search for:
Rhino-MCP-Platform
```

Restart Rhino after the package installs.

Ref: McNeel RhinoAI (https://mcneel.github.io/RhinoAI/); Anthropic Claude Desktop MCP Documentation.

Installing the Rhino MCP (mcneel.github.io/RhinoAI)

SETUP GUIDE // RHINO 8 AND CODEX

STEP 01 // ENVIRONMENT

Rhino 8

Use current Rhino 8 on an Apple Silicon Mac.

CLI / COMMAND

```
# In Rhino 8 command line:
SystemInfo
# Check the Rhino version
```

Tested here: Rhino 8.36 on Apple Silicon.

STEP 02 // PACKAGE

Install MCP Server

Install Rhino-MCP-Platform from Rhino Package Manager.

RHINO COMMAND

```
PackageManager
# Search for:
Rhino-MCP-Platform
```

Restart Rhino after the package installs.

STEP 03 // CONFIG

Configure Harness

In MCPConnect, click Install next to Codex.

CODEX CONNECTION

```
MCPConnect
```

Restart Codex to load the Rhino tools.

Ref: McNeel RhinoAI (https://mcneel.github.io/RhinoAI/); Anthropic Claude Desktop MCP Documentation.

Installing the Rhino MCP (mcneel.github.io/RhinoAI)

SETUP GUIDE // RHINO 8 AND CODEX

STEP 01 // ENVIRONMENT

Rhino 8

Use current Rhino 8 on an Apple Silicon Mac.

CLI / COMMAND

```
# In Rhino 8 command line:
SystemInfo
# Check the Rhino version
```

Tested here: Rhino 8.36 on Apple Silicon.

STEP 02 // PACKAGE

Install MCP Server

Install Rhino-MCP-Platform from Rhino Package Manager.

RHINO COMMAND

```
PackageManager
# Search for:
Rhino-MCP-Platform
```

Restart Rhino after the package installs.

STEP 03 // CONFIG

Configure Harness

In MCPConnect, click Install next to Codex.

CODEX CONNECTION

```
MCPConnect
```

Restart Codex to load the Rhino tools.

STEP 04 // EXECUTE

Verify Connection

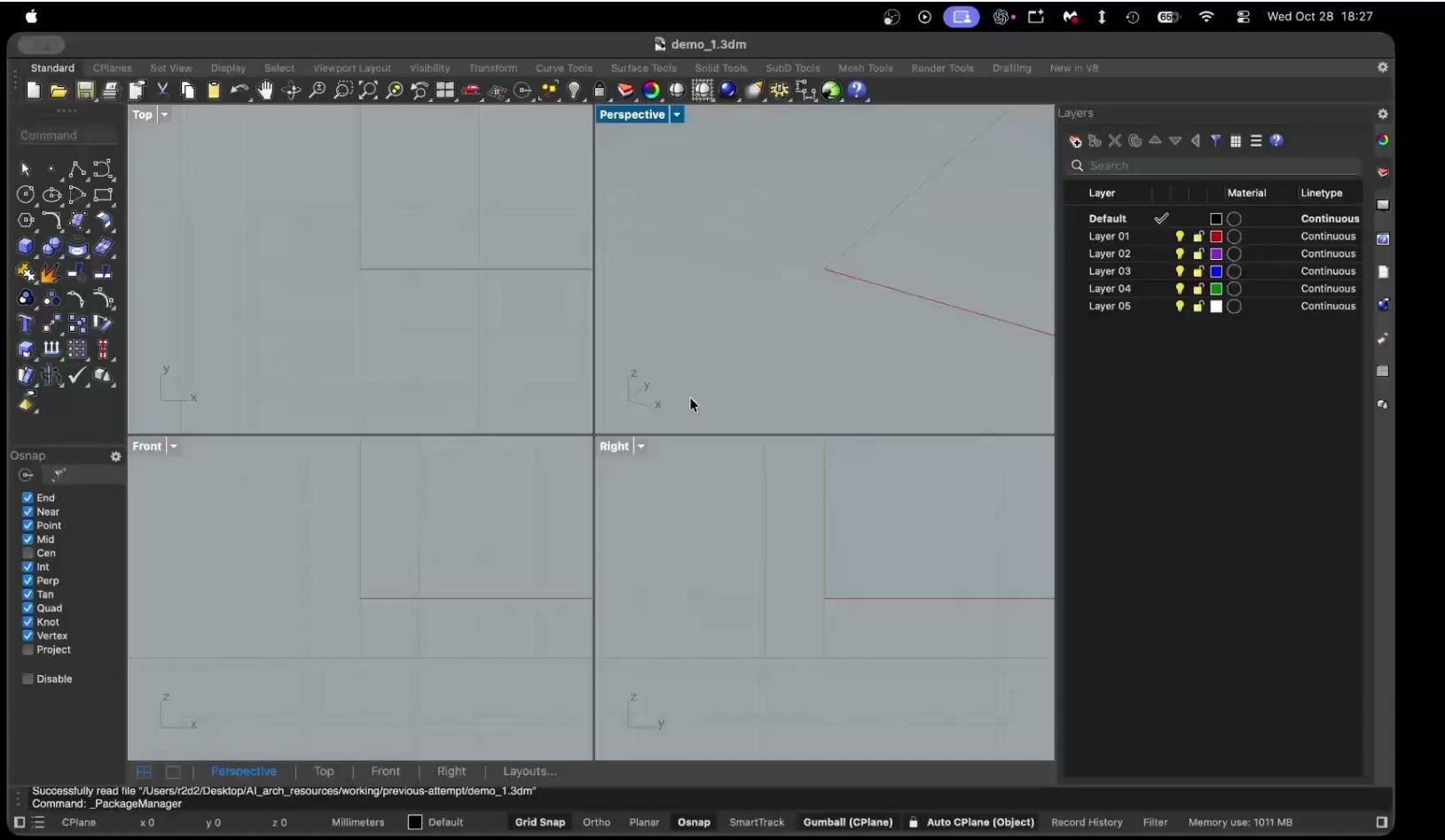
Launch MCP bridge inside Rhino and prompt the LLM.

RHINO 8 VERIFICATION

```
# Inside Rhino 8:
MCPStart
# Prompt: 'Draw a torus'
```

Check: read the model, then create one torus.

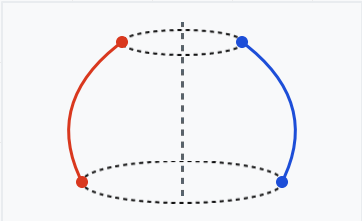
Ref: McNeel RhinoAI (https://mcneel.github.io/RhinoAI/); Anthropic Claude Desktop MCP Documentation.


Wed Oct 28 18:27
demo_1.3dm
Standard
CPlanes
Set View
Display
Select
Viewport Layout
Visibility
Transform
Curve Tools
Surface Tools
Solid Tools
SubD Tools
Mesh Tools
Render Tools
Drafting
New in V8
Command
Top
Perspective
Front
Right
Layers
Search
Layer
Material
Linetype
Default
Continuous
Layer 01
Layer 02
Layer 03
Layer 04
Layer 05
Osnap
End
Near
Point
Mid
Cen
Int
Perp
Tan
Quad
Knot
Vertex
Project
Disable
Perspective
Top
Front
Right
Layouts...
Successfully read file "/Users/r2d2/Desktop/AI_arch_resources/working/previous-attempt/demo_1.3dm"
Command: _PackageManager
CPlane
x 0
y 0
z 0
Millimeters
Default
Grid Snap
Ortho
Planar
Osnap
SmartTrack
Gumball (CPlane)
Auto CPlane (Object)
Record History
Filter
Memory use: 1011 MB

System Example 1A: Rhino MCP Command Selection

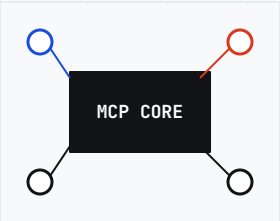
01 / INPUT GOAL

Target Form Sketch / Goal



02 / MODEL SELECTION

Rhino MCP



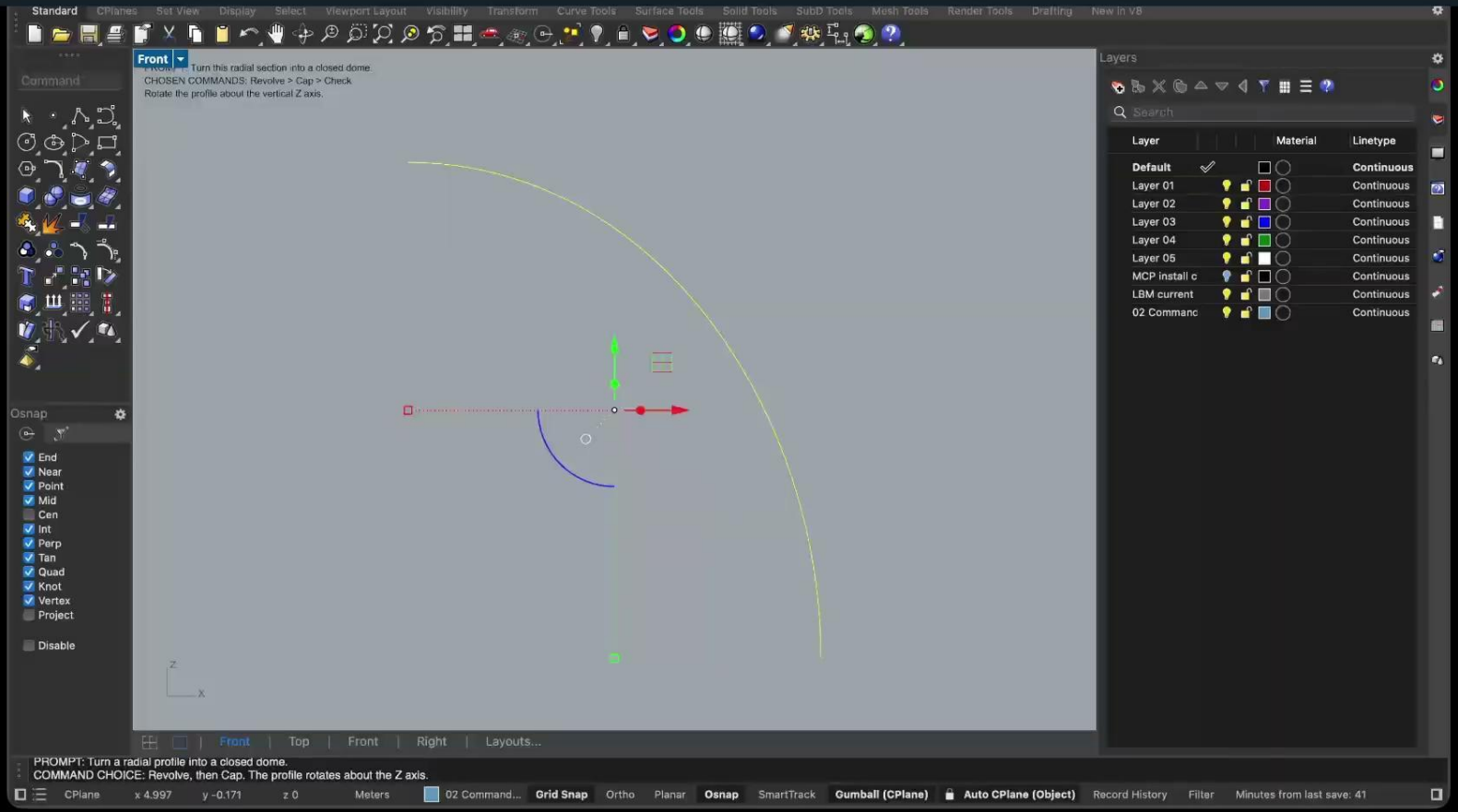
03 / RECOMMENDED COMMAND SEQUENCE

Recommended Command Sequence:

1. Radial Profile
2. Revolve
3. Cap
4. Check

Sequential command retrieval via Model Context Protocol (MCP)

Rhino commands: radial profile, Revolve, Cap, then geometry checks



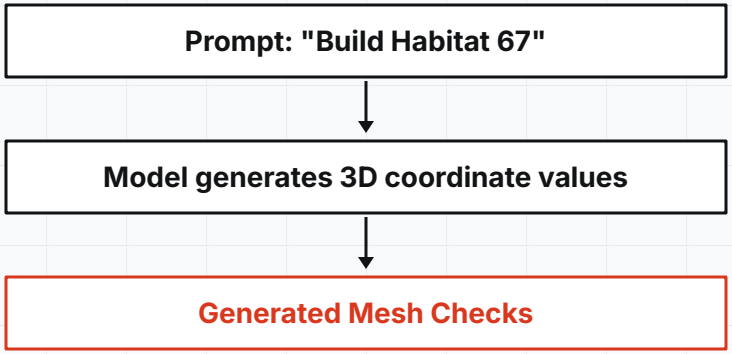
Habitat 67





Habitat 67 - Safdie Architects - https://www.safdiearchitects.com/projects/habitat-67

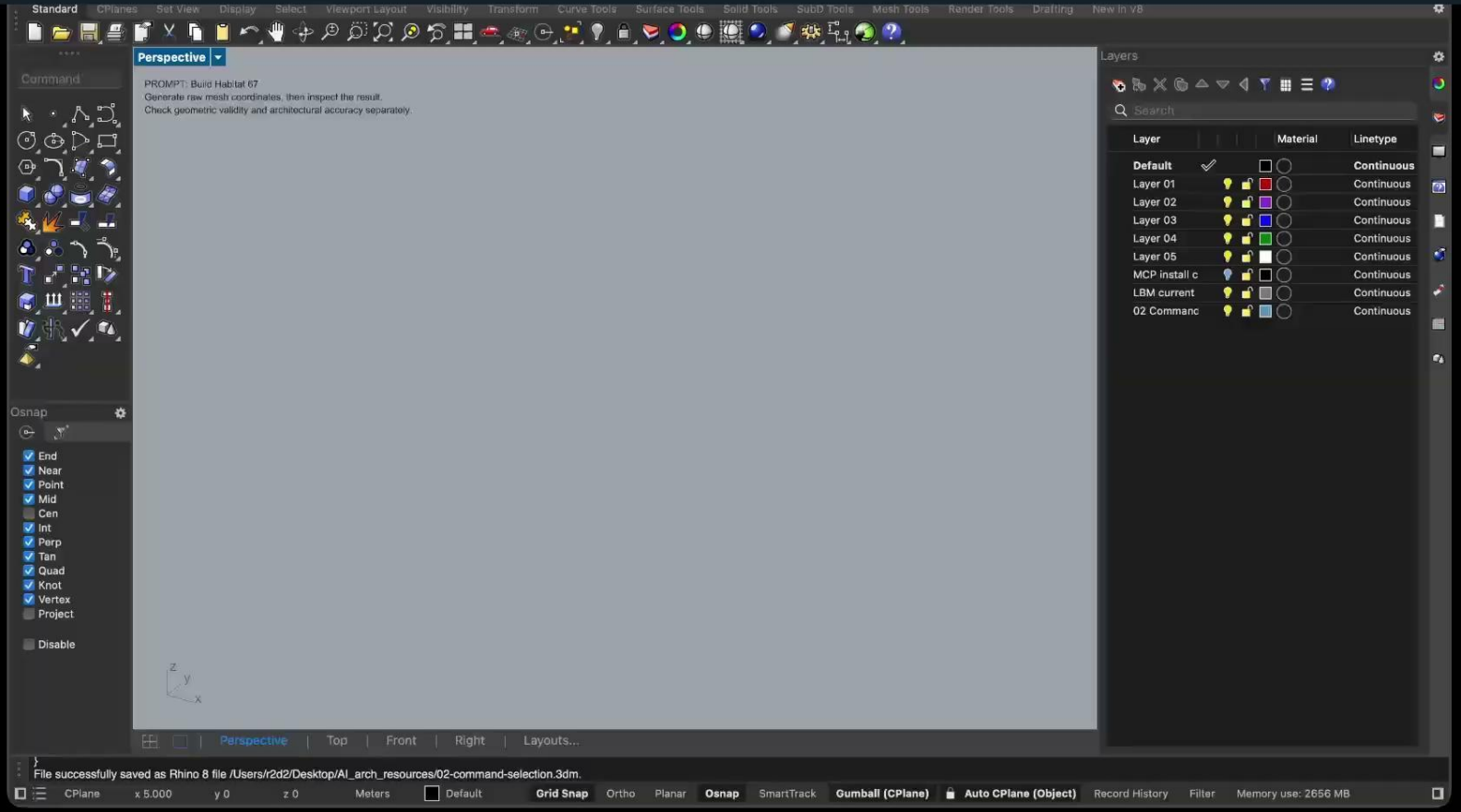
System Example 1B: Direct 3D Coordinate Output



- This run produces 126 assumed modules in one closed mesh.
- Module dimensions and the stepped layout are assumptions.
- A valid mesh still needs comparison with the building.

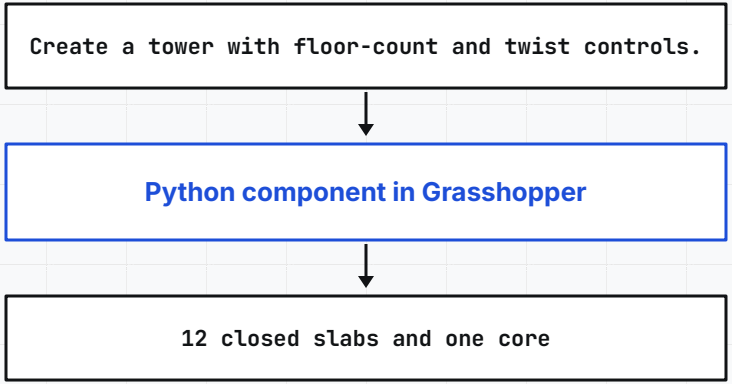
Reference: Safdie Architects, Habitat 67. Approximate massing.

Prompt: Build Habitat 67. Approximate massing from mesh coordinates



Demo 2: Grasshopper MCP

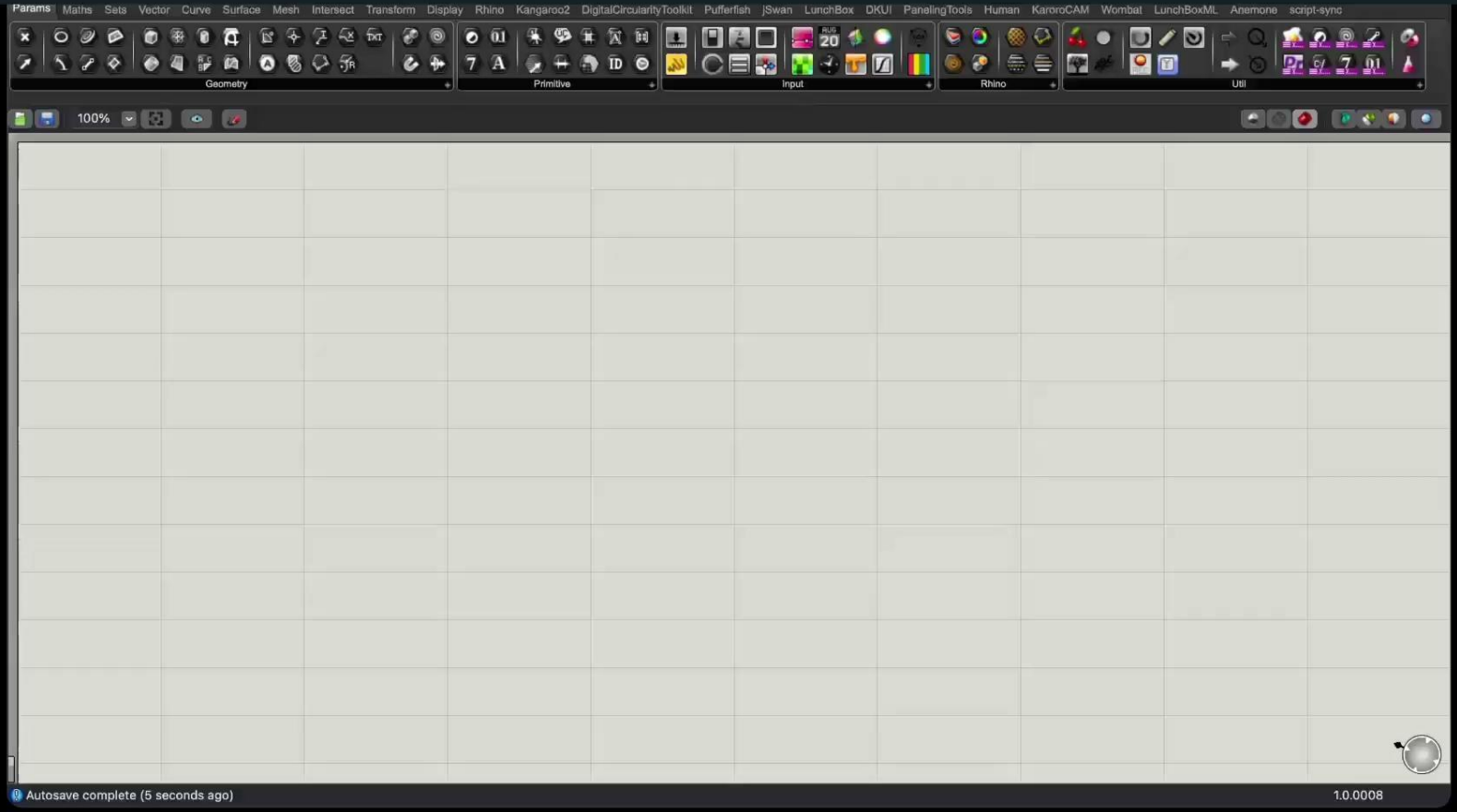
System Example 2A: Parametric Script Generation via Grasshopper



The model writes script syntax; the CAD engine executes geometric calculation.

Script generation produces parametric definitions that execute within the CAD environment according to defined geometric rules.

Build the Grasshopper definition and run its Python component



System Example 2B: Airflow in a 2D Slice

Adapt existing LBM code into a Grasshopper tool for a 2D flow slice.

01 // INPUT PARAMETERS

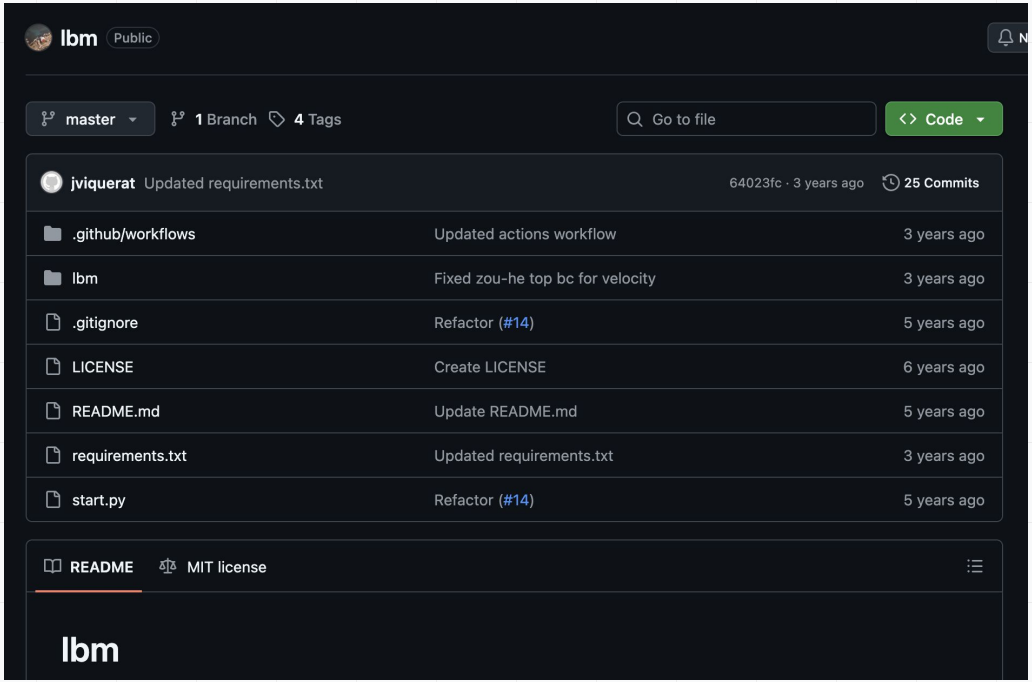
Building Slice

Closed footprints, inlet speed, grid size and viscosity.

Compare two layouts with the same flow settings.

System Example 2B: Airflow in a 2D Slice

Reference repository



https://github.com/jviquerat/lbm

System Example 2B: Airflow in a 2D Slice

Adapt existing LBM code into a Grasshopper tool for a 2D flow slice.

01 // INPUT PARAMETERS

Building Slice

Closed footprints, inlet speed, grid size and viscosity.

Compare two layouts with the same flow settings.



02 // CODE ADAPTATION

Adapt Existing Code

- Ports D2Q9 / TRT from GitHub
- Runs a small Python solver

```
// LBM time step
for each step {
collide_and_stream()
apply_boundaries()
}
```

System Example 2B: Airflow in a 2D Slice

Adapt existing LBM code into a Grasshopper tool for a 2D flow slice.

01 // INPUT PARAMETERS

Building Slice

Closed footprints, inlet speed, grid size and viscosity.

Compare two layouts with the same flow settings.

02 // CODE ADAPTATION

Adapt Existing Code

- Ports D2Q9 / TRT from GitHub
- Runs a small Python solver

```
// LBM time step
for each step {
collide_and_stream()
apply_boundaries()
}
```

03 // GENERATED TOOL

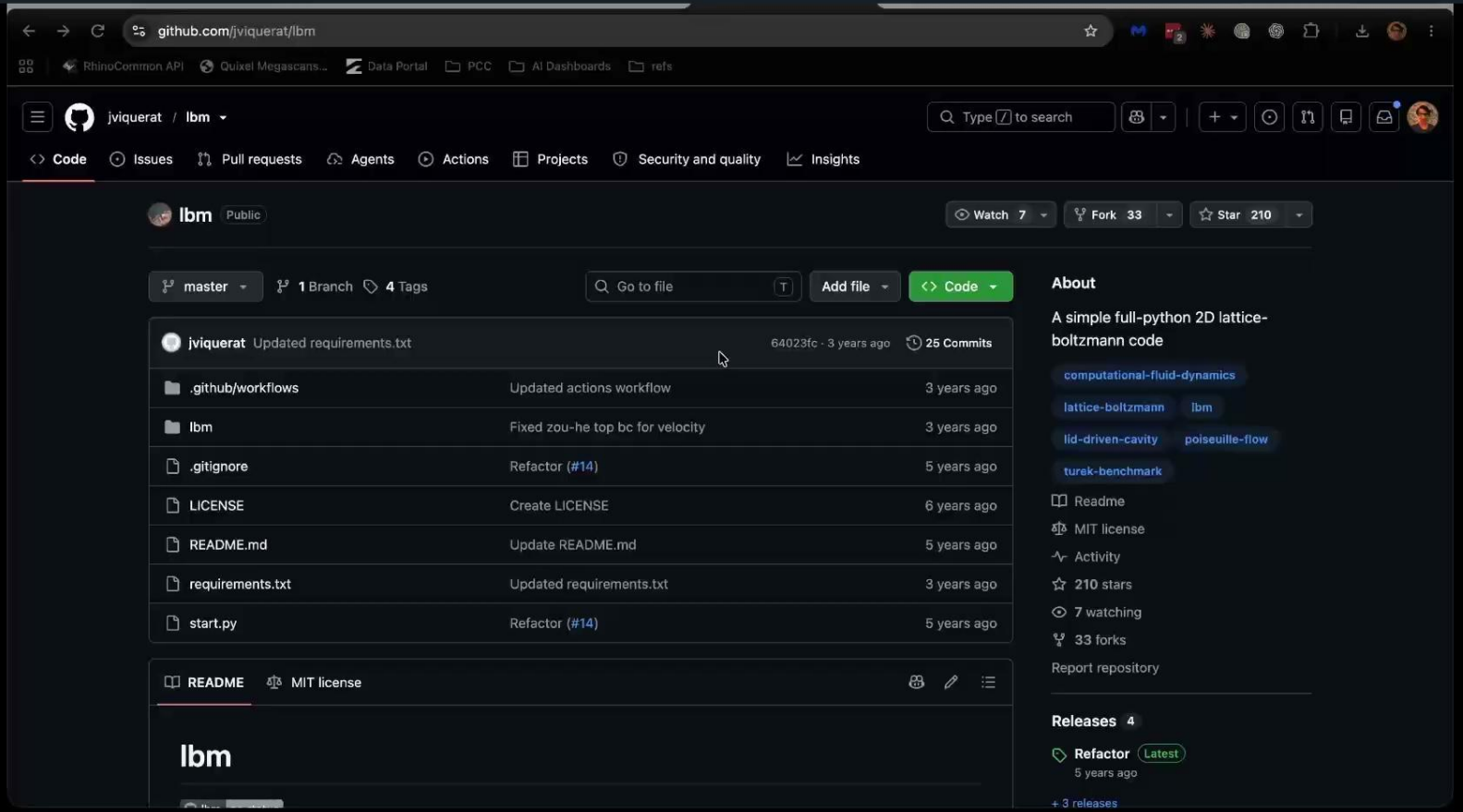
2D Flow Field

Teaching model. Check stability before comparing layouts.

OUTPUT METRICS:

- Relative speed
- Velocity vectors
- Wake pattern

1. Inspect the GitHub source: jviquerat/lbm, D2Q9 / TRT (2x)



System Example 2C: Extracting Regulatory Constraints for Grasshopper Models

Function: Models extract quantitative constraints from regulatory texts to define parametric building envelopes.

[INPUT 01]

Toronto RD Zoning Rules

Selected Toronto clauses.
Fictional 15 x 36 m lot.

Format: Unstructured PDF / Text

Process: Regulatory Document → Parameter Extraction → Parametric Boundary Generation

System Example 2C: Extracting Regulatory Constraints for Grasshopper Models

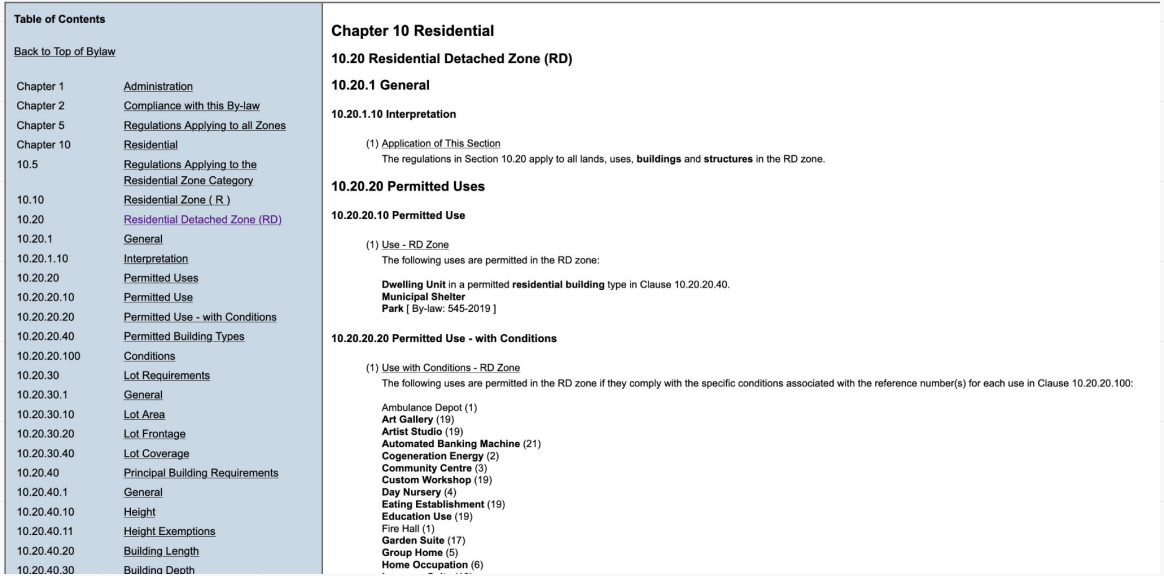


Table of Contents

Back to Top of Bylaw

Chapter 1	Administration
Chapter 2	Compliance with this By-law
Chapter 5	Regulations Applying to all Zones
Chapter 10	Residential
10.5	Regulations Applying to the Residential Zone Category
10.10	Residential Zone (R)
10.20	Residential Detached Zone (RD)
10.20.1	General
10.20.1.10	Interpretation
10.20.20	Permitted Uses
10.20.20.10	Permitted Use
10.20.20.20	Permitted Use - with Conditions
10.20.20.40	Permitted Building Types
10.20.20.100	Conditions
10.20.30	Lot Requirements
10.20.30.1	General
10.20.30.10	Lot Area
10.20.30.20	Lot Frontage
10.20.30.40	Lot Coverage
10.20.40	Principal Building Requirements
10.20.40.1	General
10.20.40.10	Height
10.20.40.11	Height Exemptions
10.20.40.20	Building Length
10.20.40.30	Building Depth

Chapter 10 Residential

10.20 Residential Detached Zone (RD)

10.20.1 General

10.20.1.10 Interpretation

(1) Application of This Section

The regulations in Section 10.20 apply to all lands, uses, **buildings** and **structures** in the RD zone.

10.20.20 Permitted Uses

10.20.20.10 Permitted Use

(1) Use - RD Zone

The following uses are permitted in the RD zone:

Dwelling Unit in a permitted **residential building** type in Clause 10.20.20.40.
Municipal Shelter
Park [By-law: 545-2019]

10.20.20.20 Permitted Use - with Conditions

(1) Use with Conditions - RD Zone

The following uses are permitted in the RD zone if they comply with the specific conditions associated with the reference number(s) for each use in Clause 10.20.20.100:

Ambulance Depot (1)
Art Gallery (19)
Artist Studio (19)
Automated Banking Machine (21)
Cogeneration Energy (2)
Community Centre (3)
Custom Workshop (19)
Day Nursery (4)
Eating Establishment (19)
Education Use (19)
Fire Hall (1)
Garden Suite (17)
Group Home (5)
Home Occupation (6)

https://www.toronto.ca/zoning/bylaw_amendments/ZBL_NewProvision_Chapter10_20.htm

System Example 2C: Extracting Regulatory Constraints for Grasshopper Models

Function: Models extract quantitative constraints from regulatory texts to define parametric building envelopes.

[INPUT 01]

Toronto RD Zoning Rules

Selected Toronto clauses.
Fictional 15 x 36 m lot.

Format: Unstructured PDF / Text

[DATA PROCESSING 02]

Text Parsing

Check each clause. Declare the zone, setbacks and coverage assumptions.

Structured Data Extraction

Process: Regulatory Document → Parameter Extraction → Parametric Boundary Generation

System Example 2C: Extracting Regulatory Constraints for Grasshopper Models

Function: Models extract quantitative constraints from regulatory texts to define parametric building envelopes.

[INPUT 01]

Toronto RD Zoning Rules

Selected Toronto clauses.
Fictional 15 x 36 m lot.

Format: Unstructured PDF / Text

[DATA PROCESSING 02]

Text Parsing

Check each clause. Declare the zone, setbacks and coverage assumptions.

Structured Data Extraction

[OUTPUT 03]

Grasshopper 3D Model

Change lot depth to 30 m.
Recompute setbacks and house size.

3D Parametric Volume

Process: Regulatory Document → Parameter Extraction → Parametric Boundary Generation

1. Read Toronto RD clauses. State the fictional lot assumptions (2x)

toronto.ca/zoning/bylaw_amendments/ZBL_NewProvision_Chapter10_20.htm

RhinoCommon API | Quixel Megascans... | Data Portal | PCC | AI Dashboards | refs

2025]

(D) despite (A) to (B) above, the permitted maximum floor space index regulations do not apply to a **townhouse** or an **apartment building** with 60 **dwelling units** or less located on a **lot** abutting a **major street**. [By-law: 66-2024; 1062-2025(OLT); 608-2024]

10.20.40.50 Decks, Platforms and Amenities

(1) Platforms at or Above the Second Storey of a Detached House

In the RD zone, a platform such as a deck or balcony with access from the second **storey** or above of a **detached house** must comply with the following:

(A) there may be no more than a total of four platforms, and no more than one on each of the front, rear and each side of the **detached house**; and

(B) the maximum area of each platform is 4.0 square metres.

(2) Interpretation of Platform Walls

In the RD zone, if an area is not subject to **lot coverage**, in addition to regulation 10.5.40.50(1) the exterior sides of a **lawfully existing** platform that was **lawfully** enclosed in compliance with the former City of Toronto By-law 438-86 are not **main walls**.

(3) Platforms at or Above the Second Storey of a Detached Houseplex

In the RD zone, platforms such as a deck or balcony, with access from the second **storey** or above of a **detached houseplex** must comply with the following:

(A) there may be no more than a total of 2 platforms for each **dwelling unit**, and no more than one on each of the front and rear sides of the **dwelling unit**; and

(B) a platform permitted in (A) above for a **detached houseplex** on a **corner lot** may also be located on a **main wall** of a **dwelling unit** facing a **street**. [By-law: 648-2025]

10.20.40.70 Setbacks

(1) Minimum Front Yard Setback

If regulation 10.5.40.70(1) does not apply, the required minimum **front yard setback** in the RD zone is 6.0 metres.

(2) Minimum Rear Yard Setback

The required minimum **rear yard setback** in the RD zone is the greater of:

(A) 7.5 metres; or

(B) 25% of the **lot depth**.

(3) Minimum Side Yard Setback

The required minimum **side yard setback** in the RD zone is:

(A) 0.6 metres if the required minimum **lot frontage** is less than 6.0 metres;

(B) 0.9 metres if the required minimum **lot frontage** is 6.0 metres to less than 12.0 metres;

(C) 1.2 metres if the required minimum **lot frontage** is 12.0 metres to less than 15.0 metres;

(D) 1.5 metres if the required minimum **lot frontage** is 15.0 metres to less than 18.0 metres;

(E) 1.8 metres if the required minimum **lot frontage** is 18.0 metres to less than 24.0 metres;

(F) 2.4 metres if the required minimum **lot frontage** is 24.0 metres to less than 30.0 metres; and

(G) 3.0 metres if the required minimum **lot frontage** is 30.0 metres or greater.

(4) Shifting Minimum Side Yard if Required Lot Frontage is in Specified Range

Despite regulation 10.20.40.70(3), for a **lot** in the RD zone with a required minimum **lot frontage** of 12.0 metres to less than 18.0 metres, the required minimum **side yard setback** on one side of a permitted **residential building**, may be reduced by a maximum of 0.3 metres if the required minimum **side yard setback** on the other side of the permitted **residential building** is increased by the same amount. [By-law: 474-2023]

(5) Larger Minimum Side Yard Beyond Specified Depth if Required Lot Frontage is Over 18.0 Metres

Despite regulation 10.20.40.70(3), for a **lot** in the RD zone with a required minimum **lot frontage** greater than 18.0 metres, the required minimum **side yard setback** is 7.5 metres for any portion of

Demo 3: Autodesk Forma

System Example 3: Autodesk Forma (Published Demo)

Building massing, type and material

↓

Forma Building Layout Explorer

↓

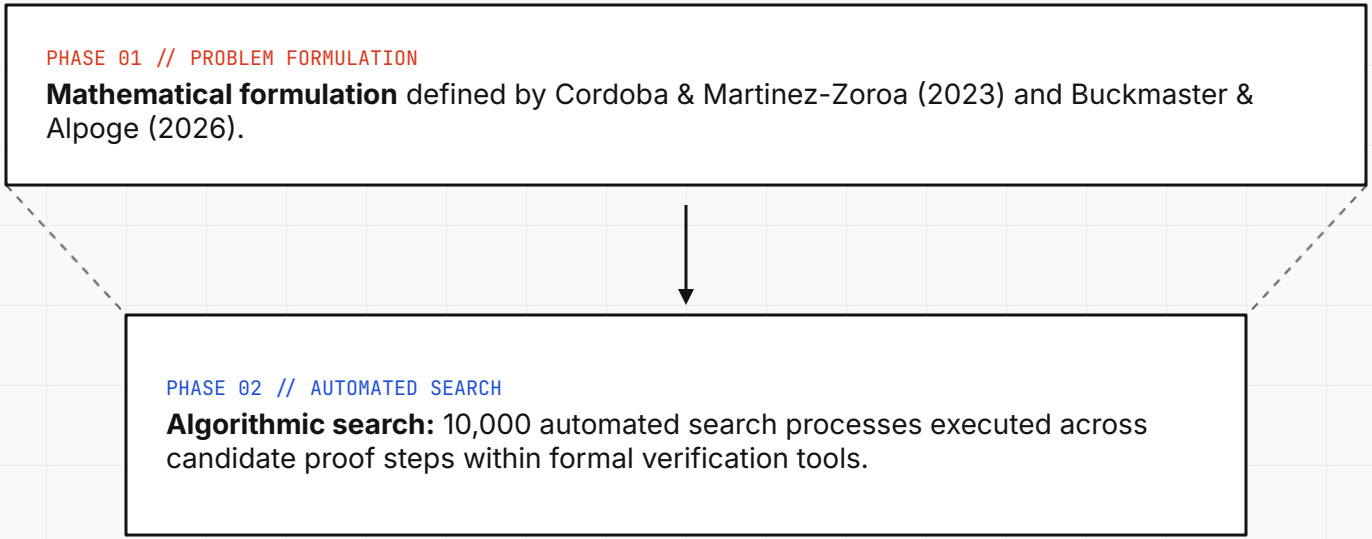
Floor-plan options (experimental)





4. Broader questions about AI

Computational Optimization: Navier-Stokes Case Study



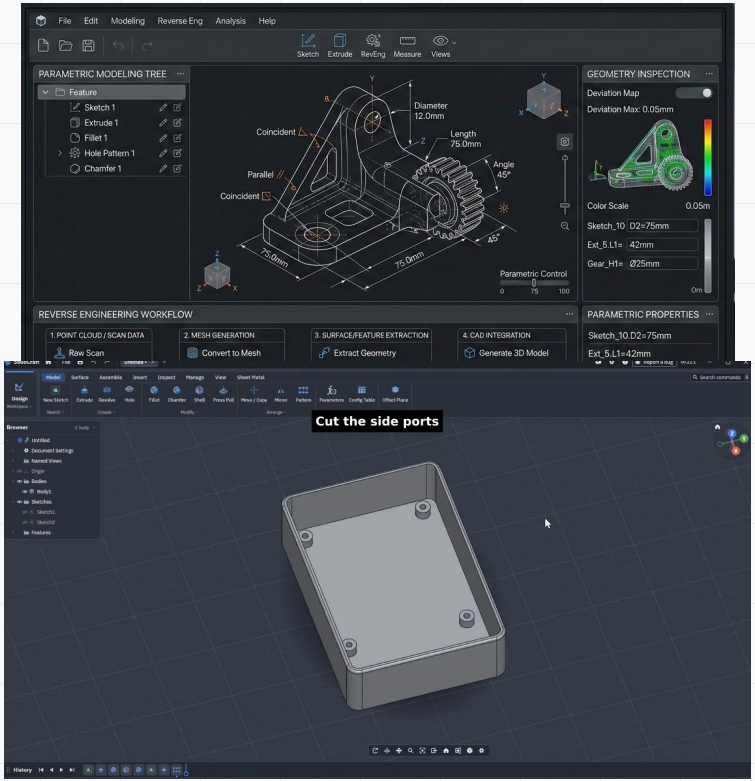
Ref: Cordoba & Martinez-Zoroa (2023); Buckmaster & Alpoge (2026); OpenAI (2026).

Model Utility: General-Purpose Optimization and Retrieval

Functional profile: Models perform automated iterative candidate generation and validation within defined constraints.

CAD Code Reconstruction: CadCraft and PartCraft

- **Parametric Code Generation:** CadCraft and PartCraft generate CAD script commands to reconstruct specified 3D target geometry.
- Closed-Loop Geometry Verification: Model generates script → CAD kernel executes geometry → shape discrepancy is calculated → script updates until geometric tolerance is reached.



Reference Material and Technical Documentation

VIDEO LECTURES // VISUAL MATHEMATICS

3Blue1Brown: Neural Networks and Deep Learning

A video lecture series explaining matrix operations, gradient backpropagation, and geometric transformations in high-dimensional coordinate spaces.

- **Topics:** Weights, biases, activation functions, gradient descent, and loss surfaces.
- **Application:** Visual presentation of linear algebra and vector calculus operations.

Link to Series on YouTube: 3Blue1Brown Neural Networks Playlist
youtube.com/playlist?list=PLZHQObOWTQDM4E-dwvbnQTiyKDO-y9T2t

Reference Material and Technical Documentation

MODEL ARCHITECTURE // REFERENCE CODE

Andrej Karpathy: Neural Networks: Zero to Hero

Code walkthrough showing implementation of basic neural networks and transformer models using Python and PyTorch.

Resource: karpathy.ai / YouTube

Reference Material and Technical Documentation

INTERACTIVE TOOLS // WEB VISUALIZATION

Transformer Explainer and Distill.pub

Web-based interactive diagrams demonstrating attention weight matrices, token transformations, and dimensional projections.

Resource: poloclub.github.io/transformer-explainer

Questions and Answers



AI in Architecture

Appendix A: Word Vectorization Mathematics

Mathematical projection of discrete symbolic tokens into continuous coordinate spaces.

01 // ONE-HOT ENCODING

Sparse Vector Representation

For vocabulary size |V| (e.g. 50,000 words), each token corresponds to an indicator vector:

$x_i = [0, 0, ..., 1, ..., 0]^T \in \{0, 1\}^{|V|}$

Geometric Properties:

All distinct one-hot vectors are mutually orthogonal:

$x_i \cdot x_j = 0 \quad (\text{for all } i \neq j)$

This representation assigns equal geometric distance between all distinct word pairs.

02 // EMBEDDING MATRIX

Linear Matrix Lookup

An embedding matrix $W_e \in \mathbb{R}^{(d \times |V|)}$ projects sparse indicator vectors into d-dimensional coordinate space (e.g. d = 768 or 4096):

$e_{word} = W_e \cdot x_{word} = W_e[:, index]$

Coordinate Properties:

Each token corresponds to a coordinate vector of real values:

$e_{studio} = [+0.42, +0.85, -0.19, ...]$

Embedding matrix values are adjusted during training via gradient descent.

03 // DISTANCE METRIC

Cosine Distance Metric

Vector orientation similarity is calculated using normalized dot products:

$\cos(u, v) = (u \cdot v) / (||u|| \, ||v||)$

Vector Operations:

Vector translations correspond to relational shifts:

$v_{king} - v_{man} + v_{woman} \approx v_{queen}$

In domain-specific vocabulary spaces:

$v_{studio} - v_{residential} + v_{commercial} \approx v_{office}$

Appendix B: Loss Functions and Parameter Optimization

Quantifying token prediction error and updating parameter values via gradient descent.

01 // LOSS FUNCTION

Cross-Entropy Loss (L)

The model outputs raw logits z. The softmax function calculates predicted probability values:

`p_i = exp(z_i) / Σ_j exp(z_j)`

Negative Log-Likelihood:

For target token index y_true, the error value is defined as:

`L = -log( p_target )`

- If p_target = 0.99 → L ≈ 0.01 (Low error value)
- If p_target = 0.01 → L ≈ 4.60 (High error value)

02 // CHAIN RULE OF CALCULUS

Gradient Calculation (∂L/∂W)

Backpropagation applies the chain rule to calculate the partial derivative of loss with respect to parameter W:

`∂L / ∂W_l = (∂L / ∂a_l) · (∂a_l / ∂z_l) · (∂z_l / ∂W_l)`

Gradient Flow:

Gradients propagate backward from output logits through attention and feed-forward layers to the input embedding matrix.

03 // PARAMETER UPDATES

Gradient Descent Optimization (AdamW)

In standard gradient descent, parameters update proportionally to the negative gradient scaled by learning rate η:

`W ← W - η · (∂L / ∂W)`

AdamW applies three adjustments:

1. First moment vector: tracks the exponential moving average of gradients.
2. Second moment vector: divides steps by gradient variance for adaptive per-parameter updates.
3. Weight decay: applies proportional parameter shrinkage to regularize weights.

Appendix C: Reinforcement Learning Environments in Model Training

Training feedback methodologies: human feedback (RLHF), model-based feedback (RLAIF), and deterministic execution verification (RLVR).

TYPE 1 // HUMAN PREFERENCE

RLHF: Human Feedback

Procedure:
Human annotators evaluate and rank model outputs. A reward model r_θ(x, y) is fitted to these rankings to score candidate text.

Optimization:
The policy π_θ updates to maximize reward score while limiting divergence from the base model:

```
max E[ r(x,y) - β·D_KL(π_θ || π_ref) ]
```

Characteristics: Requires human labor; subject to inter-annotator variance.

TYPE 2 // MODEL-ASSISTED PREFERENCE

RLAIF: Model Feedback

Procedure:
An auxiliary evaluation model scores and ranks candidate outputs according to specified criteria.

Iterative Refinement:
Models run automated critique loops to identify output inconsistencies without manual ranking data.

Characteristics:

Automated scaling; operates continuously without manual annotation bottlenecks.

TYPE 3 // VERIFIABLE EXECUTION

RLVR: Deterministic Execution Rewards

Procedure:
Candidate solutions are verified using external compilers and test runners:

```
r = 1 (Passes compiler and test assertions)
r = 0 (Execution error or incorrect test result)
```

Verification Environments:

- Formal proof checkers (LEAN, Isabelle)
- Code execution sandboxes and test suites
- Parametric geometry kernels and compilers

Supports search over reasoning traces with deterministic evaluation.